

Transport layer:

- End to end delivery [Port to port]

- uses TCP & UDP protocol

↓
better

↓
worse

TCP → connection oriented

→ reliable

→ Inorder delivery

→ no loss of data

→ error control, congestion control, flood control.
(checksum)

UDP → not reliable

→ datagram entity is sent to other side.

→ loss of data

→ Fast.

⇒ • main role of Transport layer is to provide communication services directly to app. process at diff. hosts.

- Provides a logical connection, both the app. process running on different host, they are not physically connected but get connected by logical connection from transport layer.

- The transport layer protocols are implemented in end systems (computers) but not in routers.

- TCP & UDP — Transport layer protocols.

- All transport layer protocol provide multiplexing and demultiplexing.

Date ___/___/___

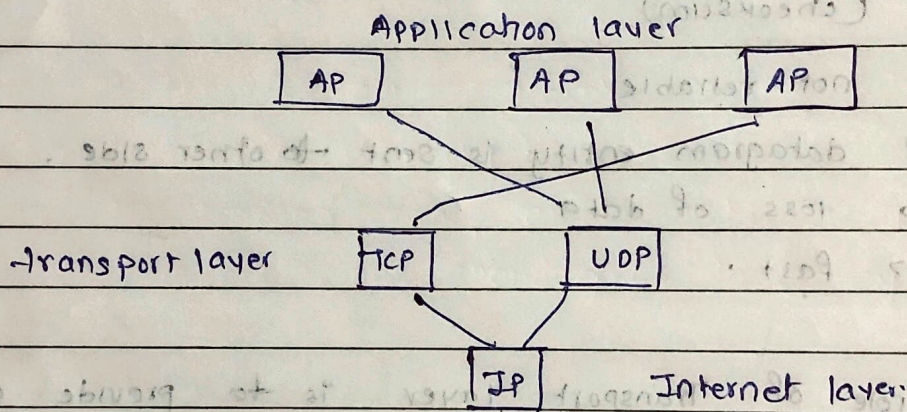
- * Each of applications in app. layer has ability to send msg by using TCP or UDP.

- App. can communicate using TCP & UDP.
- App. can read & write in transport layer, thus communication is a two way process.

① App layer (App) communicate with TCP / UDP.

they can send msg using TCP / UDP.

② TCP / UDP further communicates to Internet layer



* Services Provided by ~~TCP~~ T.L:

- Similar to services of datalink layer...

- (E A R F M)
- ① End to end delivery.
 - ② Addressing
 - ③ Reliable
 - ③ Flow control, error & congestion control.
 - ④ Multiplexing.

- * data link layer also provides node to node error control.
- * error will not get caught if error is given in betn routers.

Saathi

Date ___/___/___

• End to end delivery:

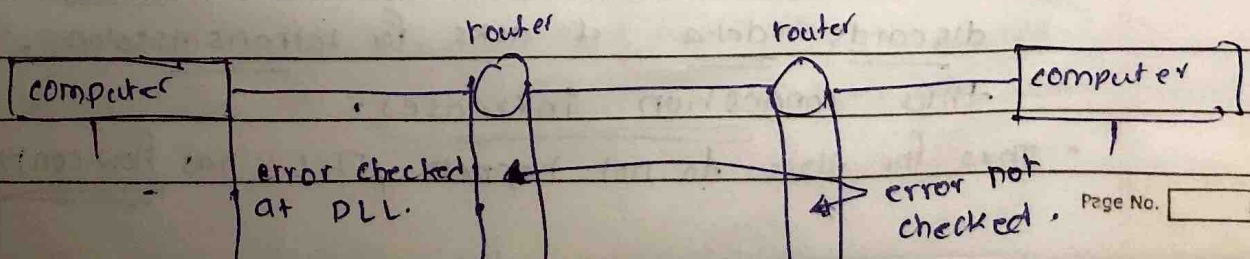
- Transmits the entire data to the destination.
thus ensures end to end delivery.

• Reliable delivery:

- provides reliability over lost packets by retransmitting
- we get,
 - Error control [ESLD]
 - sequence control
 - loss control [DELS]
 - Duplication control

* Error Control:

- transport layer protocol are designed to provide error free transmission.
- Data link layer also provides error handling mechanism, but it checks for error only at node-to-node.
So Data link layer check for error in each network,
But if error is introduced in routers, error will not get caught in data link layer.
- Data link layer checks error only at start & end.
But Transport layer checks for error, end to end
thus, packet is arrived correctly.



* seq. no. are used to identify missing & duplicates.

Saathi

Date ___/___/___

* Sequence Control :

• Thus on the sending end, transport layer is responsible for packets from upper layer ~~etc~~ can be used by lower layers.

• on receiving end, the segments sent are reassembled properly.

* Loss Control :

• It ensures all segments arrive at the destination properly.

• on sending end, all segments are given sequence numbers, thus allowing us to identify the missing.

* Duplication Control :

• no duplicates arrive at receiving end.

• sequence no. are used to identify the duplicates.

Flow Control

• If receiver is overloaded & sender sends in more segments / data, then the receiver discards data & asks for retransmission, thus congestion increases.

• Thus for this to not happen, T.L. has flow control.

sliding window protocol → handles data properly for transmission & handles flow control.
→ byte oriented than frame oriented.

Date ___/___/___

Saathi

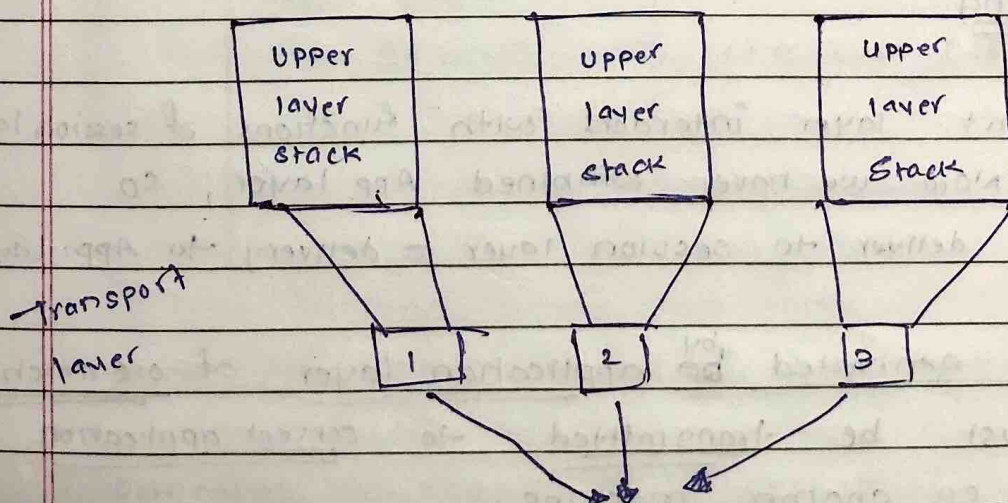
- T.L. uses sliding window protocol, helps in smooth data transfer. + controls the flow of data.

* Multiplexing → T.L. supports multiplexing.

- Multiplexing of 2 types ① upward multiplexing
② downward.

① Upward Multiplexing:

- multiple transport layer connection use same one network connection. = multiplexing
(many to one)
- to make more cost efficient;
T.L. sends several transmission bound for same destination along same path.



one virtual circuit

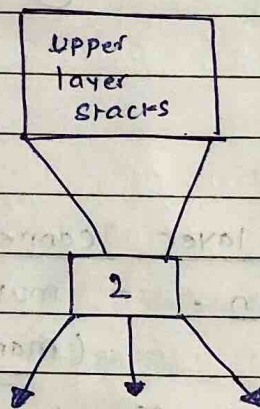
- * upward multiplexing ⇒ virtual circuit has 2 path. Upward
⇒ several transmission on same path along same path.

Downward → 1 to many (T.L. connection, net. L. connection)
→ T.L. connec. splits into several parts, increase accuracy
upward → many to one (T.L. connectⁿ, net. L. connectⁿ)
→ many T.L. connectⁿ to same destⁿ are sent through same path → cost efficiency.

Saathi

② Downward Multiplexing:

- One transport layer connection uses multiple network layer connections.
- allows transport layer to split into a connection among several paths to improve accuracy.



← one T.L. connectⁿ.

Transport layer.

Downward

one Virtual Circuit.

Addressing

- ① Transport layer interacts with functions of session layer.
- ② ~~Now~~ Now we have combined App layer, so any deliver to session layer = delivery to App. layer.

* * So data generated by application layer of one machine must be transmitted to correct application on another machine. here addressing is used.

* T.L. provides the port. Port represent a particular transport service (TS) user of a point called TSAP, transport service access point.

* T.L. needs to know, which upper layer protocols are communicating.

- * IP protocol in network layer delivers datagram from source host to destination host
- * Port = 16 bits

Date / /

Saathi

Protocols of T.L ① UDP ② TCP.

① UDP → user datagram Protocol.

* msg oriented

- It is a simple protocol & provides no sequential transport of data.
- connectionless protocol.
- used when reliability & security is less important.
- end-to-end T.L. protocol, includes transport level address, Checksum error control, & information of data from upper layer.
 ^ total length.
- packet produced by UDP = datagram.

* user datagram Format (16 byte header).

Source port address (16 bit)	destination port address (16 bit)
Total length (16 bits)	checksum (16 bits)
Data	

- Source Port add. = address of app. process that delivered msg.
- Destination port add. = add. of app. process that will receive msg.
- Total length = total length of user datagram (in bytes)
- checksum = used in error ~~control~~ detection.

UDP $\rightarrow$ error detection = checksum

TCP itself segments the data

Date ___/___/___

* App. of UDP

- ① Fast
- ② Gaming, voice calls.

Saathi

• • Disadvantage of UDP: Adv = fast + sent in form of entities.

- does not provide sequencing, reordering,
- no reliability

• Data gets lost

- * • can discover error, but cannot tell which packet is lost, does not have sequence no.

② TCP $\rightarrow$ Transmission Control Protocol

- Provides full T.C. services to applications

- Connection oriented $\Rightarrow$ at first creates a connection betⁿ two ends, & generates Virtual circuit betⁿ sender & receiver for the duration of transmission.

* Features

① stream data transfer.

② Reliable

③ Flow control

④ Multiplexing

⑤ logical connection

⑥ Full duplex

* Stream data transfer: TCP transfers data in form of contagious stream of byte. It groups the byte in the form of segment & passes it to IP for transmission.

- ① when we send data, receiver send ack. of received data.
- ② if ACK not received, resend data.
- ③ when ACK contains, bytes it can handle without overflow, bytes are sent again at max no.

Date ___/___/___

Saathi

* Reliability: TCP assigns sequence to each byte transmitted + expects an true acknowledgement of bytes received from receiver TCP.

* *
• if ACK is not received in interval timeout, the TCP, resends the data.

* Flow control: so, when receiver sends ACK back to sender, of how many bytes it can receive without congestion.

• The no. of bytes sent in ACK in the form of highest sequence number, that it can receive without any problem.

⇒ Window mechanism.

* Multiplexing: process of accepting + forwarding data.

forwarding data to correct app. = demultiplexing.

* TCP sends data through ports

* logical connection: ~~see~~ combination of sockets, seq. no., window size, = logical connection.

Full Duplex = data flows in both directions at same time.

IP address = unique 32 bit string of numbers that identifies each computer using internet protocol to communicate over network.

UDP $\Rightarrow$

Source	destination
Total length	checksum
data	

Saathi

Date ___/___/___

UDP $\rightarrow$ 8 bytes

TCP segment $\rightarrow$ 20 bytes

Source port address (16 bits)								Destination port address (16 bits)							
Sequence no (32 bits)															
Acknowledgement no (32 bits)															
HLEN	Reserved	U	A	P	R	S	F	window size (16 bit)							
4 bits	6 bits	R	C	S	S	Y	J								
		G	K	H	T	N	N								
Check sum (16 bits)								Urgent pointer (16 bits)							
X								X							
options & padding															

- * Source port add = add. of app. prog. of source computer.
- * Sequence no = stream of data is divided into segments, seq. no. represents ~~data~~ position of data in ~~data~~ stream.
- * ACK no. = ack. data from other communicating devices.
If ACK = 1, receiver is expecting to receive.
- * Reserved = reserved for future.
- * Window size = defines size of window in flow control.
- * Checksum = error detection.
- * Urgent ptr = if URG = 1, then urgent ptr is offset from seq. no. indicating that it is last urgent data byte.
- * Opt & padding = convey additional info. to receiver.

- * HLEN $\rightarrow$ size of tcp header. in 32 bit words
min size = 5 words max = 15 words.

full duplex $\Rightarrow$ data transmission on both sides TCP = client server.

Saathi

Date ___/___/___

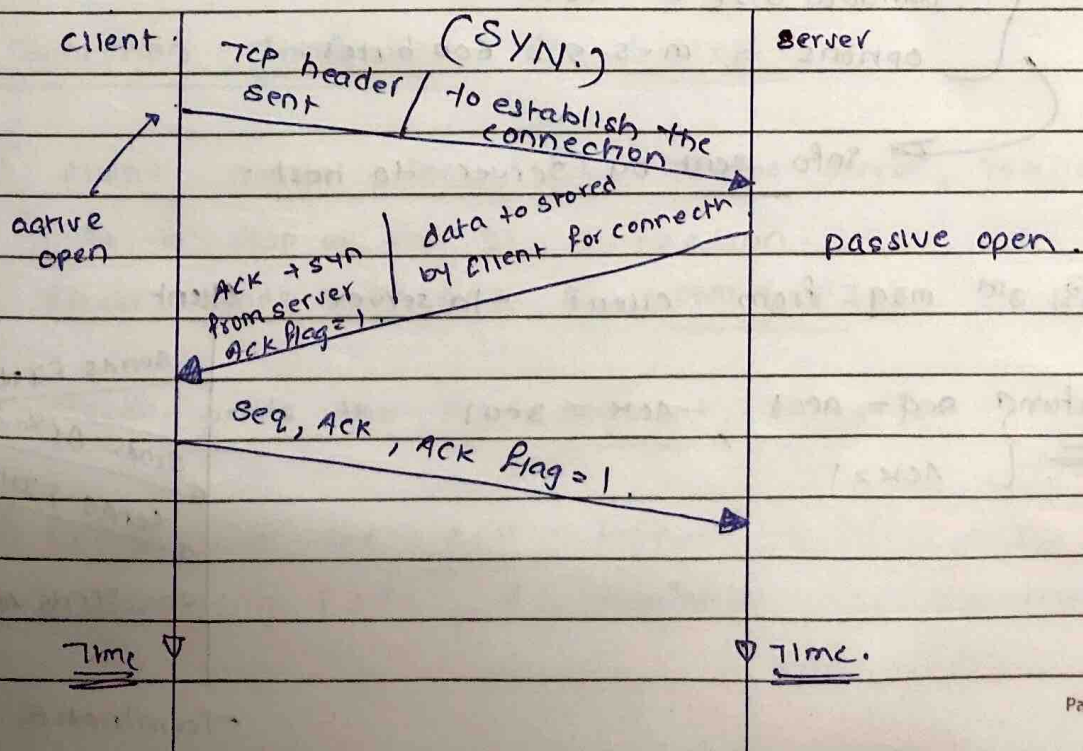
* Control bits (URG, SYN, ACK, RST, PSH, FIN).

- URG = data in segment is urgent
- ACK = if it is set, then it validates the ack no.

- * • PSH = informs that higher throughput is needed.
- RST = resets TCP connection if confusion in seq. no.
- SYN = synchronises the msg. to 3 parts, request, confirmation & acknowledgement.
- FIN = sender has finished sending data.

TCP connection establishment

- connection establishment is performed by using 3 way handshaking. This syncs both ends of the network & helps agree on eg. seq. no.
- mechanism also provides both the sides to transmit data & know if other side is able to communicate.



$$\text{seq. no} = 0 - (2^{32} - 1)$$

$$\text{ACK} = \text{seq} + 1$$

Saathi

Date ___/___/___

- ① 1st msg ^{sent} to establish the connectn.
data is not transmitted.

port = A, Seq = 9000.

SYN = 1, to synchronises with server.

send the window size = 12000.

options = max segment size = 1200 byte.

→ we give this data in 1st segment sent, to establish connection.

* now server stores this info.

- ② 2nd msg from server to client,
server will give its info. along with
syn + ACK (your packet received).

port no = 80, dest. port = A

Seq = 3000, ACK 9001

ACK = 1, SYN = 1.

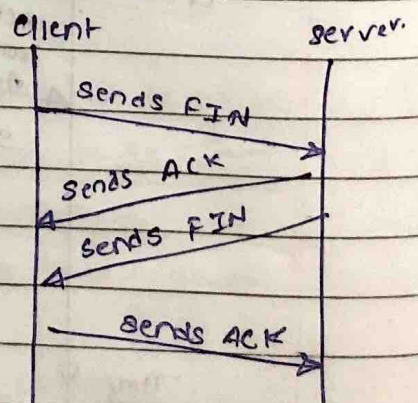
Window size = 18000.

options = m.s.s = 600 bytes.

→ info sent by server to host.

- ③ 3rd msg from client to server.

confirmations Seq = 9001, ACK = 3001
ACK = 1



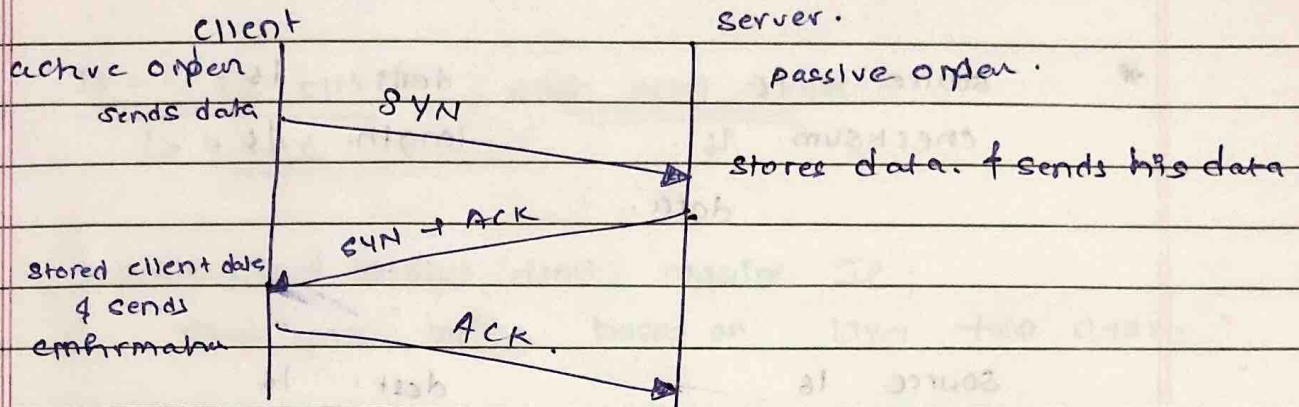
Size of TCP header = ~~40 bytes~~ = 20 bytes

Flags $\Rightarrow$ ACK, URG, RST, PSH, FIN, SYN

UDP header size = 8 bytes.

Date ___/___/___

Saathi



Data transfer:

- Once connection is established, bidirectional data transfer is done.

- msg is divided in 4 segments.

$\rightarrow$ first three sends both data & acknowledgement
& last sends only acknowledgement

- When receiver gets msg / packets, it increases the ack by (length) of received data.

Connection Termination $\rightarrow$ 4 steps.

- ① client sends the FIN to the server, requesting it to stop or end the connection.
- ② server sends the ACK of receiving FIN.
- ③ server sends the FIN segment, ready for termination.
- ④ client sends the ACK segment to server, for receiving FIN, & terminates the connection.